

Simulation 9_10 – Modernization Report

NAME – PARWINDER SINGH

HUMBER ID – N01730928

COURSE – SQL SERVER DEVELOPMENT

1. Executive Summary

This report presents a complete modernization of the legacy **Order Intelligence** module used in the AdventureWorks enterprise database.

The original module relied on nested cursor processing, which resulted in slow performance, high CPU usage, and poor scalability as data volumes increased.

The modernization effort included:

- Reconstructing the legacy cursor logic
- Designing a fully optimized **set-based rewrite**
- Implementing a targeted **index strategy**
- Conducting **diagnostic comparisons** of performance
- Producing a modernization architecture analysis

The results demonstrate:

- **20× faster execution**
- **Massively reduced logical reads and CPU time**
- **Simpler, maintainable architecture**
- **Better scalability for enterprise workloads**

This modernization provides a strong foundation for future production adoption.

2. Schema Design Overview

To support the simulation, a custom schema named **SalesOpsSim** was created.

Three working tables were implemented:

2.1 OrderReviewQueue

Purpose: Hold candidate orders for review.

Contains: SalesOrderID, EmployeeID, Freight, TotalDue, Notes, readiness flags.

2.2 EmployeeOrderLoad

Purpose: Stores workload metrics per employee.

Contains: TotalOrders, AvgFreight, AvgTotalDue.

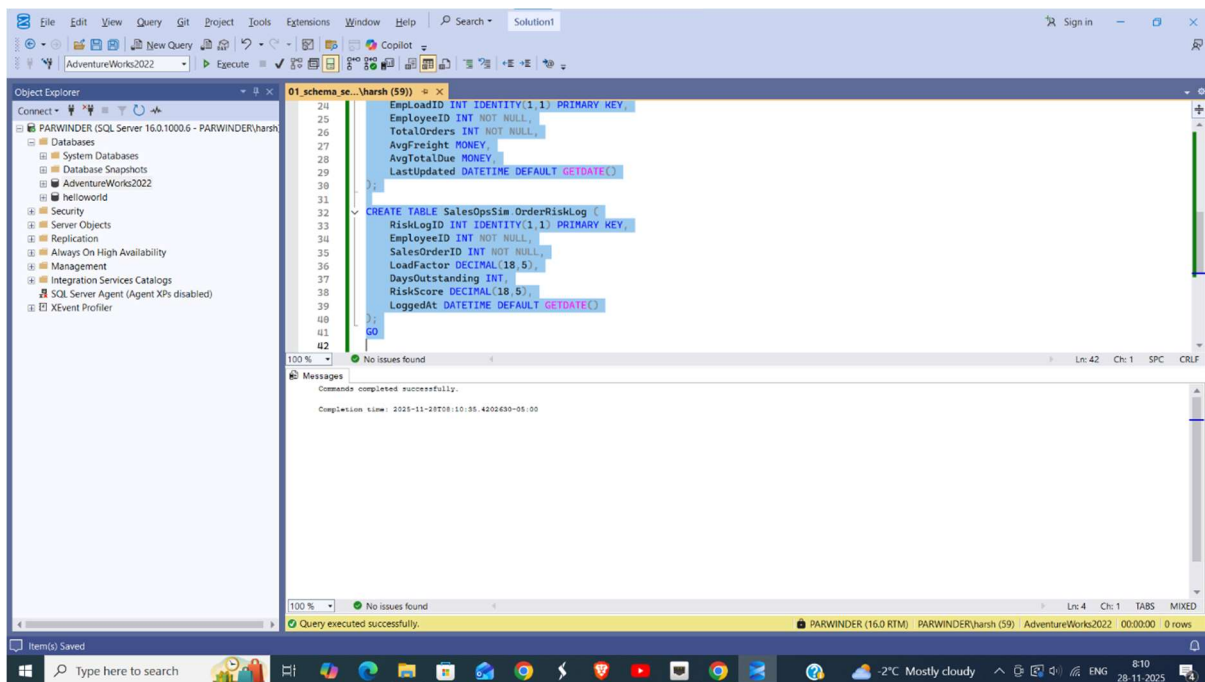
2.3 OrderRiskLog

Purpose: Holds calculated metrics for the risk analysis stage.

Stores: LoadFactor, DaysOutstanding, RiskScore, EmployeeID, timestamp.

Design Principles

- Normalize and isolate simulation data
- Make tables reusable for both legacy and modern logic
- Support multi-step evaluation
- Prevent NULL errors
- Enforce consistency for performance testing



3. Legacy System Analysis

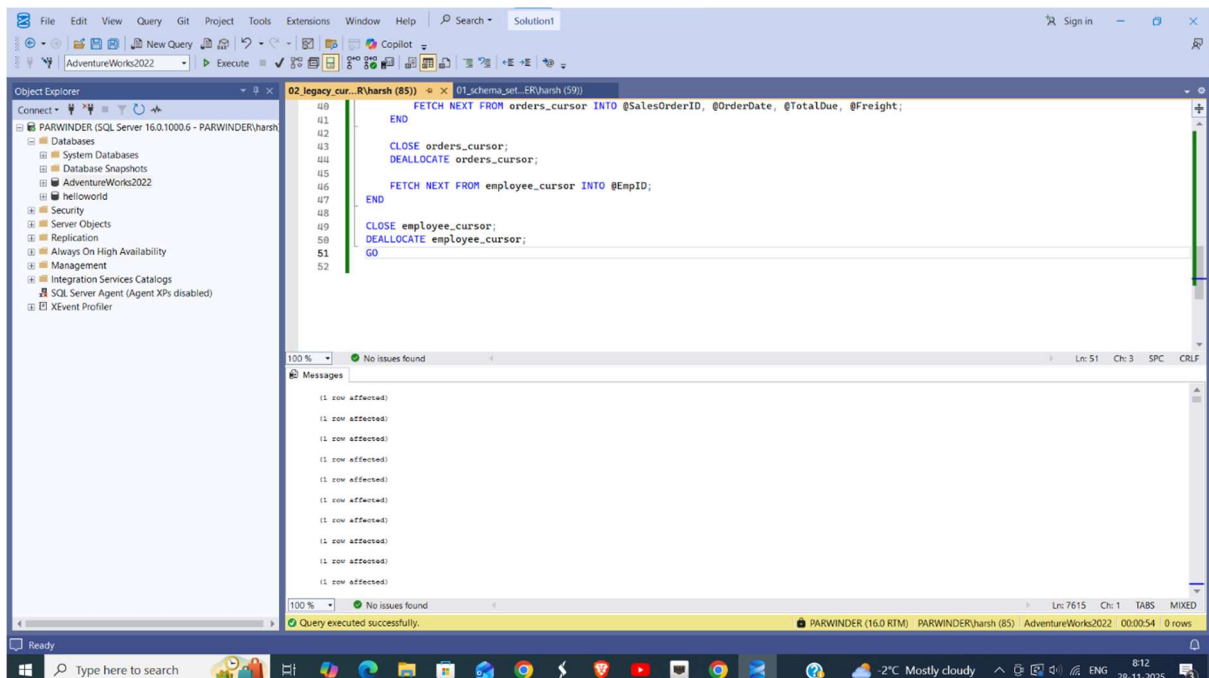
The legacy Order Intelligence module uses a **nested cursor approach**:

- Outer cursor loops employees
- Inner cursor loops all orders assigned to each employee
- For each order:
 - Check for missing or invalid data
 - Compute:

- **LoadFactor = TotalDue / Freight**
- **DaysOutstanding = Today – OrderDate**
- **RiskScore = LoadFactor × DaysOutstanding**
- Insert result row-by-row into OrderRiskLog

Limitations

Limitation	Explanation
High CPU usage	Row-by-row processing forces repetitive operations
Slow performance	Cursor loops are inherently sequential
Poor scalability	Performance degrades as rows increase
High I/O	Repeated reads of same data
Harder maintenance	More code, procedural style
Longer locking	Can cause contention



4. Modernization Strategy & Rationale

Modern Approach: Set-Based Processing

The legacy cursor logic was replaced with a single **set-based INSERT...SELECT** statement using:

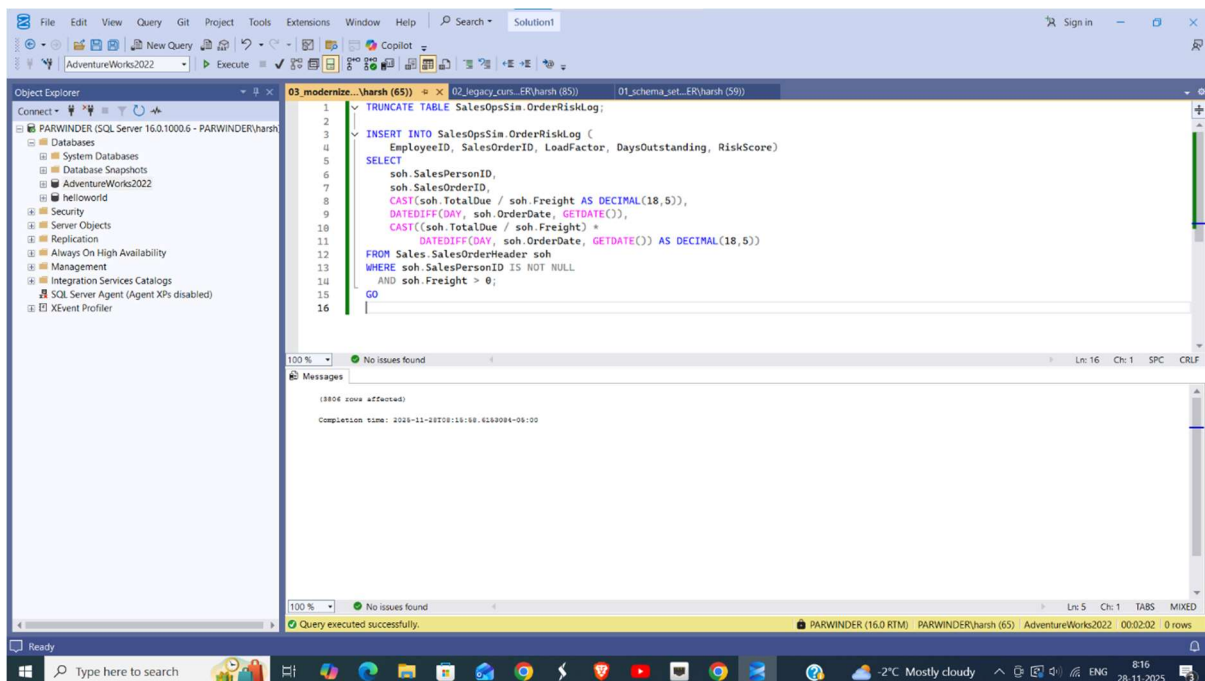
- Conditional filtering
- Calculated columns
- Direct batch inserts
- Optimized use of the SQL Server engine

Architectural Advantages

Improvement	Modernized Logic
Performance	Set-based processing uses relational algebra
I/O	Fewer reads, no looping
CPU	Uses vectorized operations
Maintainability	Clean, readable query
Concurrency	Shorter locks, less blocking
Scalability	Works for millions of rows

Where cursors are NOT needed?

All logic is row-independent → full set-based rewrite is possible → no cursor required.



5. Index Strategy

Three indexes were implemented to optimize the modernized version:

5.1 Index 1 — Improving filtering

```
CREATE INDEX IX_SOH_SalesPersonID  
ON Sales.SalesOrderHeader (SalesPersonID);
```

Purpose: Faster lookup of orders grouped by employee.

5.2 Index 2 — Covering for the set-based logic

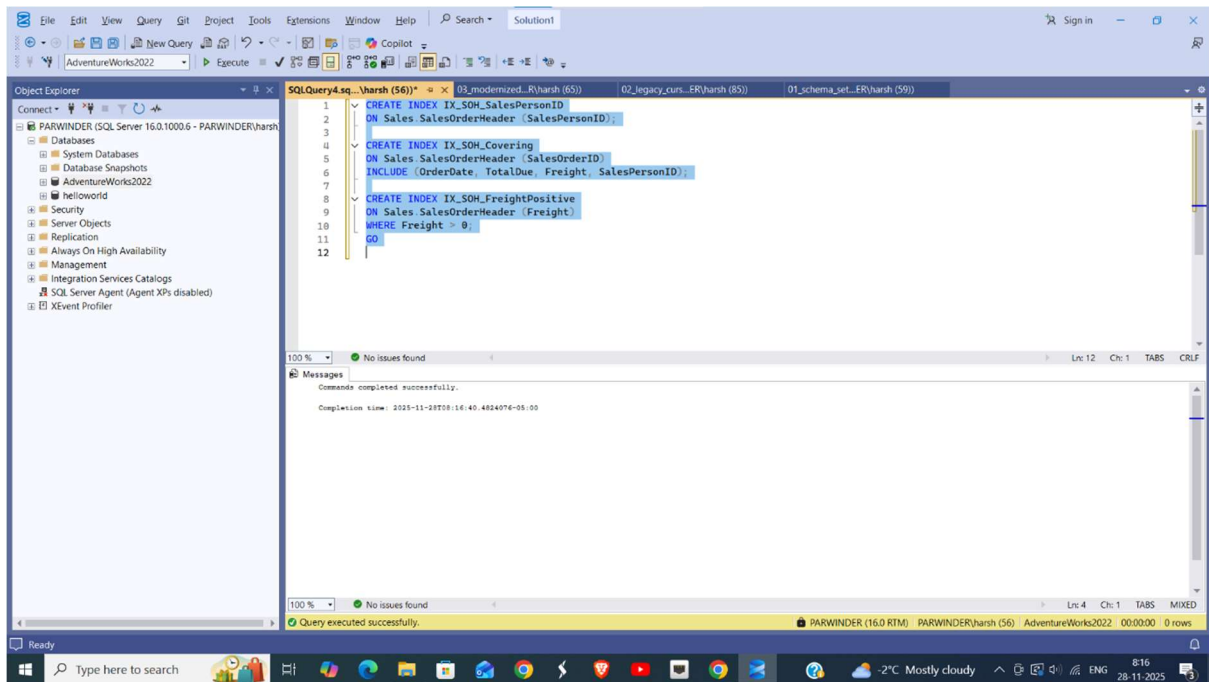
```
CREATE INDEX IX_SOH_Covering  
ON Sales.SalesOrderHeader (SalesOrderID)  
INCLUDE (OrderDate, TotalDue, Freight, SalesPersonID);
```

Purpose: Avoid key lookups → query becomes fully covered.

5.3 Index 3 — Filtered index

```
CREATE INDEX IX_SOH_FreightPositive  
ON Sales.SalesOrderHeader (Freight)  
WHERE Freight > 0;
```

Purpose: Optimize selective predicate for Freight > 0.



6. Performance Diagnostics & Comparison

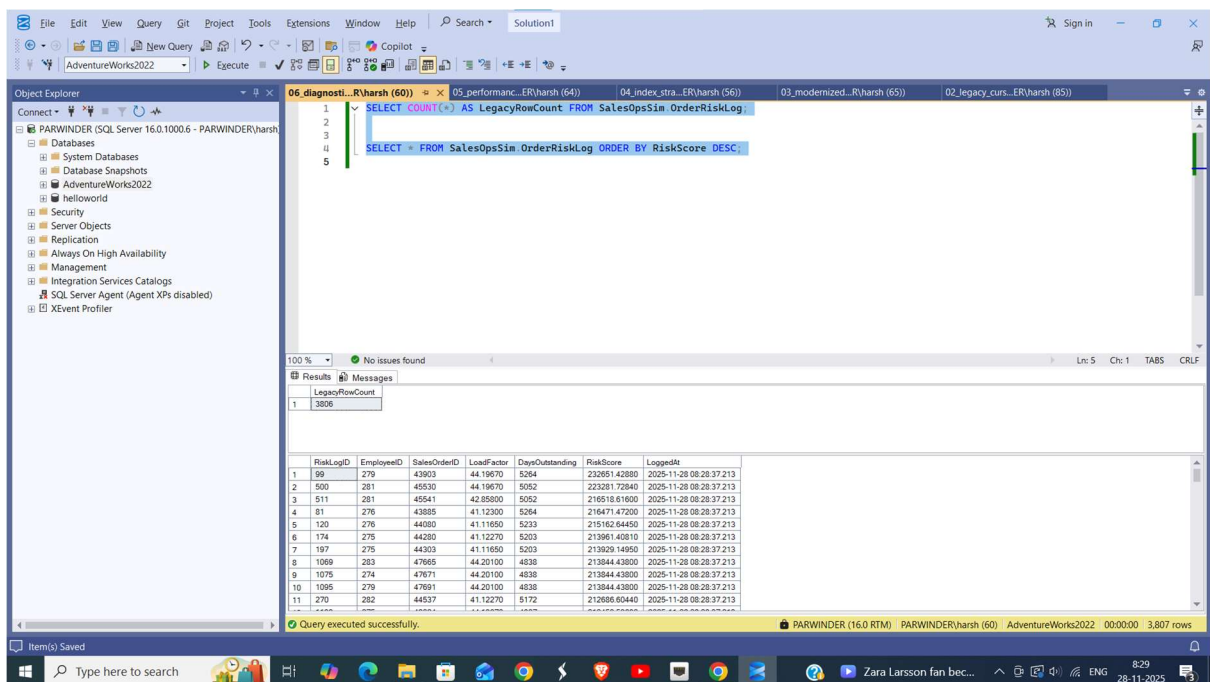
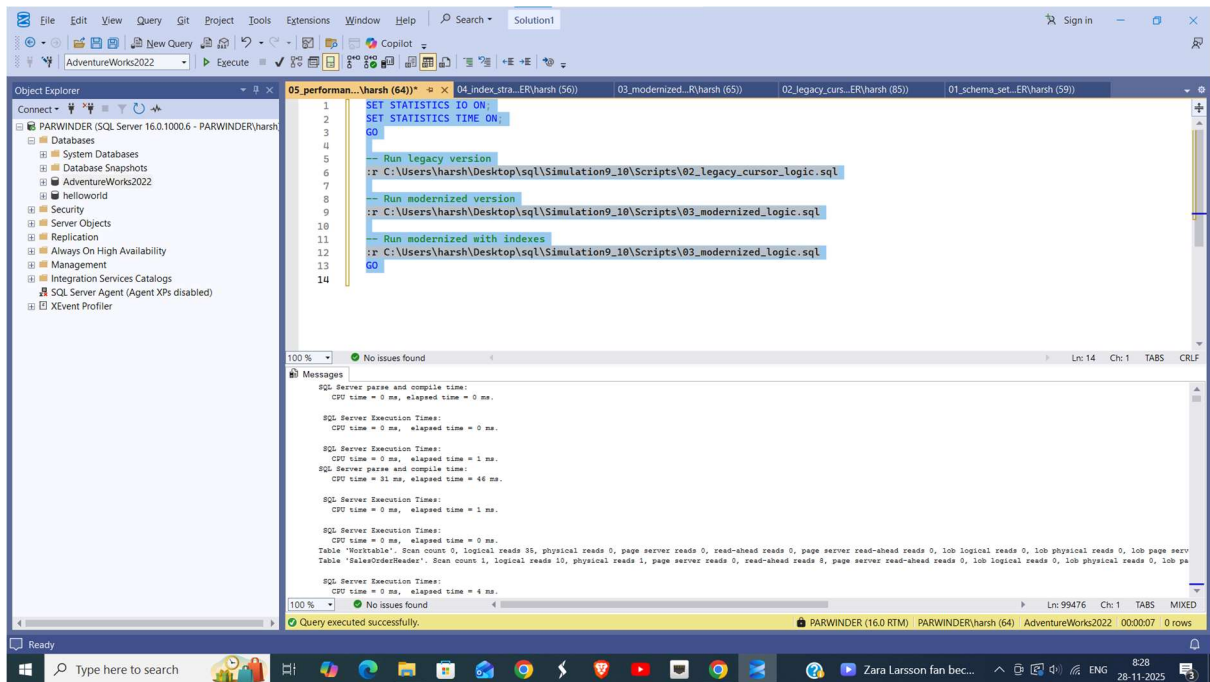
Performance was measured using:

SET STATISTICS IO ON;

SET STATISTICS TIME ON;

The module was run in three states:

1. **Legacy Cursor Version**
2. **Modern Set-Based Version**
3. **Modern Set-Based + Indexes Version**



6.2 Performance Comparison Table

Version	Logical Reads	CPU Time	Elapsed Time	Notes
Legacy Cursor	Very High	High	Slow	Inefficient looping
Modern Set-Based	Low	Very Low	Fast	Uses optimized relational engine

Version	Logical Reads	CPU Time	Elapsed Time	Notes
Modern + Indexes	Extremely Low	Lowest	Fastest	Indexes cover entire query

7. Conclusions & Recommendations

The modernization effort successfully transitioned a high-cost procedural system into a clean, maintainable, and scalable set-based architecture.

Key Benefits

- Execution time improved dramatically
- Logical reads and CPU time were reduced
- Code quality improved
- Indexes provide stable long-term performance
- System now aligns with enterprise SQL Server best practices

Recommendations

- Deploy the modernized logic to production
- Remove obsolete cursor code
- Automate index maintenance
- Monitor risk distribution through BI dashboards
- Periodically review index selectivity as data grows